# Object-Oriented Programming in Python

Comprehensive Theory + Code Examples

# Classes & Objects - Theory

- Classes are blueprints for objects
- Objects are instances of classes
- Attributes: Class vs Instance
- Methods: Instance, Class, Static

# Classes & Objects - Code

```
class Person:
    species = "Homo sapiens"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"Hi, I’m {self.name},
   {self.age} years old."

p1 = Person("Alice", 25)
print(p1.introduce())
```

# Classes & Objects - Explanation

- species → Class attribute shared by all objects
- __init__() → Constructor initializes object data
- introduce() → Instance method
- p1 is an object (instance of Person)

# Encapsulation - Theory

- Encapsulation = Binding data and methods together
- Private attributes use '__' prefix
- Controlled access via getters/setters

# Encapsulation - Code

```
class BankAccount:
    def __init__(self, owner,balance=0):
        self.owner = owner
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

# Encapsulation - Explanation

- __balance is private (not accessible directly)
- deposit() modifies balance safely
- get_balance() returns value in a controlled way

# Inheritance - Theory

- Inheritance allows reusing and extending code
- Child classes inherit methods/attributes from parent
- Can override parent methods

# Inheritance - Code Demo

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f"{self.name} makes a sound."

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks."

dog = Dog("Buddy")
print(dog.speak())
```

# Inheritance - Explanation

- Animal is the parent class
- Dog inherits from Animal
- Dog overrides speak() method

# Polymorphism - Theory

- Polymorphism: Same interface, different behavior
- Implemented via method overriding
- Works with collections of objects

# Polymorphism - Code Demo

```
animals = [Dog("Max"), Cat("Luna")]

for animal in animals:
    print(animal.speak())
```

# Polymorphism - Explanation

- Both Dog and Cat have speak()
- Same method name, different outputs
- Polymorphism makes code flexible

# Abstraction - Theory

- Abstraction hides implementation details
- Achieved via Abstract Base Classes (ABC)
- Forces subclasses to implement abstract methods

# Abstraction - Code Demo

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

rect = Rectangle(3,4)
print(rect.area())
```

# Abstraction - Explanation

- Shape is an abstract class
- area() must be implemented in subclasses
- Rectangle provides concrete implementation

# Operator Overloading - Theory

- Special methods define custom behavior for operators
- __add__, __sub__, __str__ etc.
- Example: Adding two Vector objects

# Operator Overloading - Code Demo

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector(self.x + other.x,
self.y + other.y)
    def __str__(self):
        return f"({self.x}, {self.y})"

v1, v2 = Vector(2,3), Vector(4,5)
print(v1 + v2)
```

# Operator Overloading - Explanation

- __add__ defines + operator behavior
- __str__ defines object print format
- v1 + v2 creates new Vector

# Multiple Inheritance & MRO - Theory

- A class can inherit from multiple parents
- Python resolves order using Method Resolution Order (MRO)
- super() calls parent methods in MRO order

# Multiple Inheritance & MRO - Code Demo

```
class Father:
    def skill(self): return "gardening"
class Mother:
    def skill(self): return "cooking"
class Child(Father, Mother):
    def skill(self):
        return "sports + " + super().skill()

c = Child()
print(c.skill())
print(Child.mro())
```

# Multiple Inheritance & MRO - Explanation

- Child inherits both Father and Mother
- super().skill() calls Father.skill() due to MRO
- Child.mro() shows resolution order

# Composition - Theory

- Composition = HAS-A relationship
- Class contains object of another class
- Example: Car has an Engine

# Composition - Code

```
class Engine:
    def start(self): return "Engine started"
class Car:
    def __init__(self):
        self.engine = Engine()
    def drive(self):
        return self.engine.start() + ", Car
is moving"

car = Car()
print(car.drive())
```

# Composition - Explanation

- Car contains Engine object
- Car delegates start to Engine
- Demonstrates HAS-A relationship

# Realistic Example - Theory

- Employee base class with common attributes
- Developer and Manager inherit from Employee
- Demonstrates inheritance, overriding, composition

# Realistic Example - Code

```
class Employee:
    raise_pct = 1.05
    def __init__(self, name, salary):
        self.name, self.salary = name, salary
    def apply_raise(self):
        self.salary *= self.raise_pct

class Developer(Employee):
    raise_pct = 1.10
    def __init__(self, name, salary, lang):
        super().__init__(name, salary)
        self.lang = lang

class Manager(Employee):
    def __init__(self, name, salary, emps=None):
        super().__init__(name, salary)
        self.emps = emps if emps else []
    def add_emp(self, emp):
        self.emps.append(emp)

dev = Developer("Alice",60000,"Python")
mgr = Manager("Bob",80000,[dev])
mgr.add_emp(Developer("Eve",50000,"Java"))
print(mgr.emps[0].name)
```

# Realistic Example - Explanation

- Developer overrides raise_pct
- Manager manages list of Employees
- Real-world demonstration of OOP principles

# Summary of OOP in Python

- Classes & Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Operator Overloading
- Multiple Inheritance & MRO
- Composition
- Realistic Example